

Functions & Libraries

In the last part I said we want to take that code we created, and make it into something called a function. Why we do this is so we can; Easily add to our program, have it look better, or not have to rewrite code again if we want to do something similar.

Let's turn the code we had to calculate the price of the shopping cart, into a function.

First, before we need to understand what a function is again.

- Remember in math with $y = x$, or $f(x) = x$. Notice how you need to put something in to get a result, **it needs something to do its job as a function**. That's called an argument, x is an argument.

```
person = {"name": "Bob", "balance": 100, "discount": 0.1, "cart": [{  
    "strawberry": 2 }, { "banana": 3 }]}
```

In our code we have from the previous part. WE USED `person["cart"]` to complete our instructions (Go back and look at, *what we needed to complete our code*).

- That means we NEED to have it in a function so an ARGUMENT.
> An important thing is, when changing something into function. The code isn't really going to change at all, we're using the same code.
-

```
def calculate_total(cart):
```

- This is how you create a function, you say `def` to "define" THEN, you say some word that describes what you want that action to do. Then give another describing placeholder word for the argument.
 - `cart`, is a supposed to be a placeholder to equal what you want to give it. We want to give it `person["cart"]`. `cart` is the SAME as `person["cart"]`.
 - > The last important things about functions, is that they have something called a RETURN VALUE, meaning this function EQUALS this value.
 - All that really is, whatever variable that you calculated you say `return variable`. This is just the output of the function.
-

```
def calculate_cart_total(cart):  
    total = 0  
    for item in cart:  
        for price in item.values():  
            total += price  
    return total
```

This is the ENTIRETY of our function, the only thing we changed (*If you look back to the code in the last part.*) was `person["cart"]` became `cart`, and we added something called a **return value**.

- OK, let's actually use this.

So now, whenever we want to calculate the total of the cart, all we have to do, is give this **FUNCTION** the person's cart.

```
calculate_cart_total(person["cart"])
```

- This is equal to the total of the cart, it doesn't print anything. To print it, just print it.

Now we can more cleanly **ORGANIZE** our code, let's say we have an if statement as a "*customer interaction area*". that allows you do say 1,2,3,4,5 for the options you want to chose.

- **INPUTTING 1** let's you see everything in the product catalogue.
- **INPUTTING 2** let's you add stuff to your cart.
- **INPUTTING 3** let's you see your cart total
- **INPUTTING 4** let's you pay.

If we DIDN'T have functions, we'd have to put that entire for loop inside that IF STATEMENT. Instead, we can just use ONE line to cleanly create this "*customer interaction area*" for each option.

Let's quickly do it again for the "**VIEW CART**"

```
def view_cart(cart):  
    for item in cart:  
        print(item)
```

Notice, again all we changed was `cart` is now replacing `person["cart"]`

- Again, we use this function by doing `view_cart(person["cart"])`

We don't always have to **MAKE our own functions**, Libraries are community functions we can add and use.

- Let's give EACH person an ID, we could just do 1,2,3,4 but that's not safe, we could use random numbers; that would be using a library. The issue with that is **random** numbers may product duplicates

Luckily, there is a library called **UUID**, which is used to create ID's that almost never makes duplicates.

To get a library we do

```
import library
```

The library we're grabbing from is called **UUID**, the **FUNCTION** is called **UUID4**.

```
import uuid
```

SAVE the **variable**, ADD it to the person object.

- For functions we CALL them, *that is done by doing* `object.functionname()`

```
user_uuid = uuid.uuid4()
```

ADD, the ID to our object

```
person = {"id": user_uuid, "name": "Bob", "balance": 100, "discount": 0.1, "cart": [{ "strawberry": 2 }, { "banana": 3 }]}
```

That's all for functions, the next part will cover **OBJECTED ORIENTED PROGRAMMING**, which sounds fancy and complicated. But it's just combining *functions and objects*.